

Operaciones de Aprendizaje Automático III

Destilación de LLMs

MSc. Fernando Silva



Agenda

- Alineación de LLM con Reinforcement Learning
- Destilación de LLMs

Alineación de LLM con RL

Contexto

¿Qué se alinea?

El proyecto alinea el modelo a una consigna escrita, que está en datos.py:

Resolvé el problema paso a paso, en no más de tres oraciones, y cerrá con el resultado numérico entre las etiquetas <respuesta> y </respuesta>.

Eso es todo lo que significa "alineado"

No se puede entrenar contra una consigna en castellano

Hay que traducirla a un número, y eso es recompensas.py:

la consigna pide	la recompensa mide	peso
cerrar con las etiquetas	¿está <respuesta>N</respuesta>?	0.3
resolver el problema	¿el número es el correcto?	0.5
en no más de tres oraciones	¿es corta?	0.9

Flujo

GRPO

LLM
OPs

1.- Contrato de datos
la recompensa es el código
datos.py - recompensas.py

LLM
OPs

8.- Reentrenamiento con lo
observado

LLM
OPs

7.- Compuerta
Cuatro reglas, decide sola
compuerta.py

LLM

LLM
OPs

6.- Evaluación
Juez calibrado, IC del 95%
evaluar.py - analizar.py

LLM

LLM
OPs

5.- Serving
vLLM compone al desplegar
evaluar.py

LLM
GRPO

2.- Entrenamiento
GRPO + LoRA, vLLM genera
alinear.py

LLM
OPs

3.- Trazabilidad
Miflow por corrida y semilla
registro.py

LLM
OPs

4.- Artefacto
Base + adaptador + entorno
entorno.py

datos.py

GRPO

LLM
OPs

Flujo

1.- Contrato de datos
la recompensa es el código
datos.py - recompensas.py

Item generado:

- "Enunciado"
- "Respuesta"
- "Plantilla"

Plantillas:

- "Una tienda vende {a} cajas por día durante {b} días. ¿Cuántas cajas vendió en total?", lambda a, b: a * b
- "Un depósito tenía {a} unidades y se despacharon {b}. ¿Cuántas quedaron?", lambda a, b: a - b
- "Se reparten {a} libros en partes iguales entre {b} aulas. ¿Cuántos libros recibe cada aula?", lambda a, b: a // b
- "Un curso tiene {a} estudiantes y se suman {b} más. ¿Cuántos estudiantes hay ahora?", lambda a, b: a + b
- "Cada equipo necesita {a} herramientas y hay {b} equipos. ¿Cuántas herramientas hacen falta?", lambda a, b: a * b

Resolvé el problema paso a paso, en no más de tres oraciones, y cerrá con el resultado numérico entre las etiquetas <respuesta> y </respuesta>.

Construimos dataset: CONSIGNA + Enunciado

recompensas.py

GRPO

LLM
OPs

Flujo

1.- Contrato de datos

la recompensa es el código

datos.py - recompensas.py

la consigna pide	la recompensa mide	peso
cerrar con las etiquetas	¿está <respuesta>N</respuesta>?	0.3
resolver el problema	¿el número es el correcto?	0.5
en no más de tres oraciones	¿es corta?	0.9

recompensa_formato (0, 1)

recompensa_corrección (0, 1)

recompensa_brevedad
(max(0.0, 1.0 - len(texto)/120))

base.yaml

$$\text{recompensa_total} = 0.3 * \text{recompensa_formato} + 0.5 * \text{recompensa_corrección} + 0.9 * \text{recompensa_brevedad}$$

Flujo

GRPO

LLM
OPs

LLM
GRPO

1.- Contrato de datos
la recompensa es el código
datos.py - recompensas.py

2.- Entrenamiento
GRPO + LoRA, vLLM genera
alinear.py

LLM
OPs

3.- Trazabilidad
Miflow por corrida y semilla
registro.py

LLM
OPs

4.- Artefacto
Base + adaptador + entorno
entorno.py

Flujo

1.- problemas generados
por datos.py

3.- `capturar_entorno()`
GPU, versiones, precisión y
aborta si falta GPU

2 prompts
del dataset

vLLM realiza generación
6 respuestas por prompt

12 respuestas
2 grupos de 6

Puntuación para cada
respuesta

Ventaja relativa al grupo
(R-media)/desvio

2.- alineamiento

Beta x KL
Frena contra el modelo original

Gradiente
Solo mueve el adaptador LoRA

4.- guardar el adaptador

cada 50 pasos
guarda el
adaptador y
registra las
métricas

Un adaptador por checkpoint

Historial

Las métricas en MLflow

Qué se registra en cada corrida

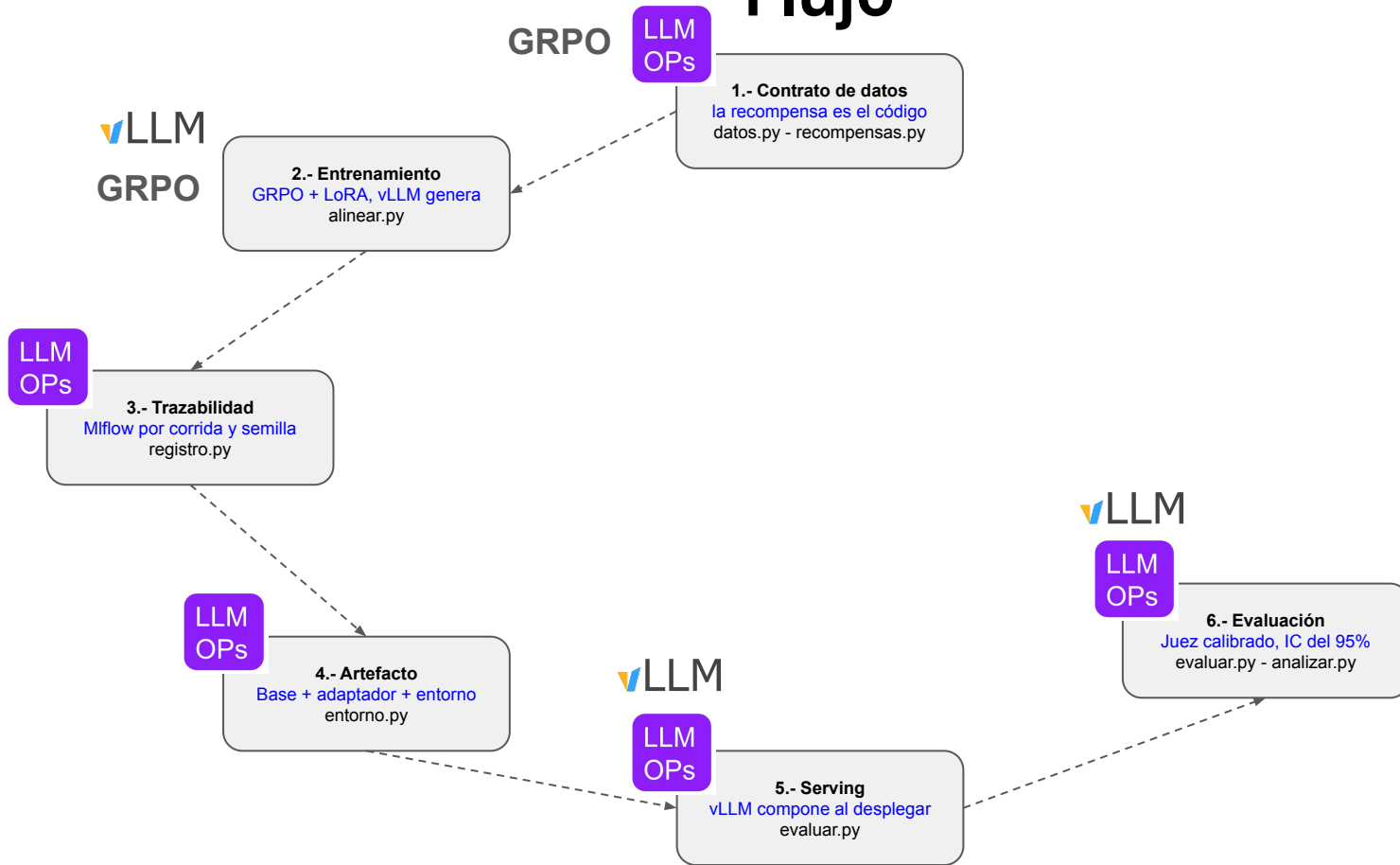
Qué	Contenido
Etiquetas	etapa, brazo, semilla, perfil. Son las que permiten filtrar las seis corridas
Parámetros	beta, el bloque de modelo, el de GRPO, el de recompensa y la huella del entorno
Métricas	Recompensa, largo, KL, clipped_ratio, frac_reward_zero_std y entropía, paso a paso
Artefactos	Los adaptadores de cada checkpoint y la figura del análisis

Los parámetros se guardan aplanados: el diccionario anidado se vuelve claves con punto.

El perfil entero queda recuperable desde el registro, sin confiar en que el YAML del repositorio no cambió desde entonces.

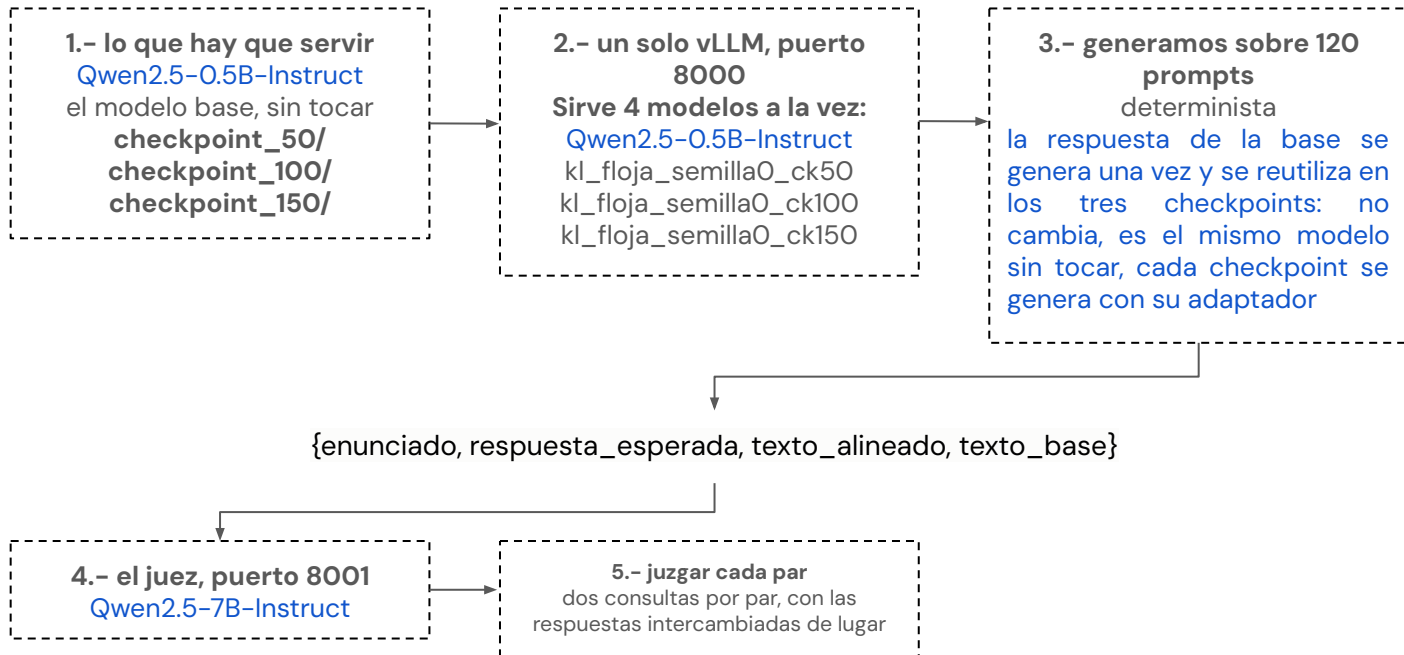
La base está en un SQLite dentro de la carpeta de artefactos. No hace falta servidor: mlflow ui lo abre local.

Flujo



evaluar.py

Flujo



Flujo

GRPO

LLM
OPs

1.- Contrato de datos
la recompensa es el código
datos.py - recompensas.py

LLM
OPs

8.- Reentrenamiento con lo
observado

LLM
OPs

7.- Compuerta
Cuatro reglas, decide sola
compuerta.py

LLM

LLM
OPs

6.- Evaluación
Juez calibrado, IC del 95%
evaluar.py - analizar.py

LLM

LLM
OPs

5.- Serving
vLLM compone al desplegar
evaluar.py

LLM
GRPO

2.- Entrenamiento
GRPO + LoRA, vLLM genera
alinear.py

LLM
OPs

3.- Trazabilidad
Miflow por corrida y semilla
registro.py

LLM
OPs

4.- Artefacto
Base + adaptador + entorno
entorno.py



Ejercicio

Alineamiento de LLMs con RL

[LINK](#)

DoRA vs LoRA

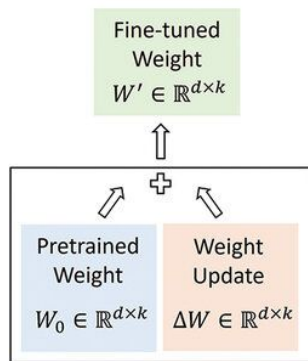
DoRA vs LoRA

LoRA aprende una actualización de los pesos mediante dos matrices pequeñas, mientras que DoRA separa esa actualización en dos partes: magnitud y dirección

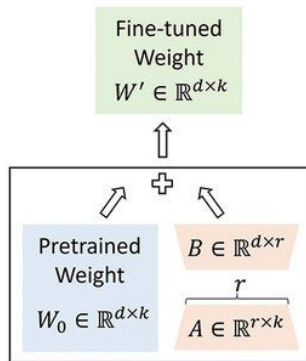
combina la magnitud aprendida con la dirección normalizada para obtener W'

introduce un parámetro entrenable m , que permite ajustar la magnitud de cada columna

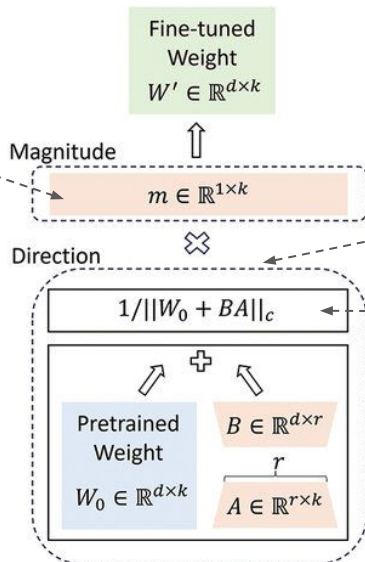
Trainable
Frozen



(a)
Normal



(b)
LoRA



(c)
DoRA

DoRA utiliza una estructura similar a LoRA, pero añade una descomposición que le permite ajustar también la magnitud de los pesos.

En lugar de considerar directamente:

$$W' = W_0 + BA$$

DoRA descompone los pesos adaptados en dos componentes:

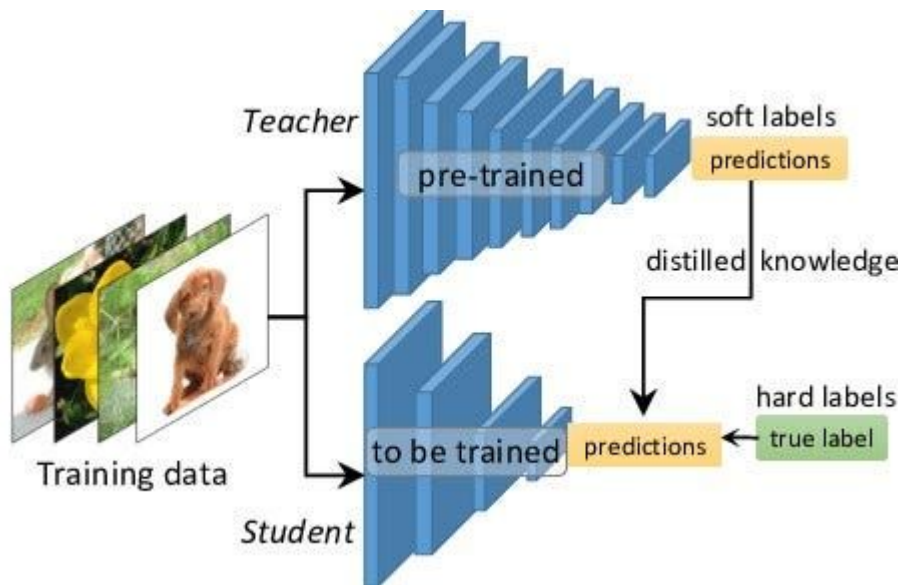
- **Magnitud:** cuánto mide el vector de pesos.
- **Dirección:** hacia dónde apunta ese vector.

El modelo puede aprender por separado si un conjunto de pesos debe tener mayor o menor magnitud y cómo debe cambiar su dirección, esto le da más flexibilidad que la parametrización de LoRA

Destilación

Destilación

Knowledge Distillation (destilación de conocimiento) es una técnica en la que entrenamos un modelo pequeño, llamado Student, para que aprenda a imitar el comportamiento de un modelo más grande y potente, llamado Teacher



- No comparamos necesariamente solo si ambos acertaron o fallaron, sino también cómo distribuyen sus probabilidades entre las posibles respuestas.
- Por ejemplo, ante una imagen de un perro, el profesor podría asignar estas probabilidades:
 - Perro: 80 %
 - Lobo: 15 %
 - Gato: 5 %

El estudiante puede aprender no solo que la respuesta correcta es perro, sino también que lobo es más parecido que gato. Esa información adicional es parte del conocimiento que se intenta transferir.



Ejercicio

Destilación de conocimiento con DoRA

[LINK](#)